Photo Storage Specification

Daniil Buchko

November 2024

1 Problem statement

Photo Storage is a web application for the private photo storage. It allows users to upload their images to the website, keep them safely, and access them in any time from any device which have access to the Internet. Besides typical features that other similar applications have, our application stores a description for every image, furthermore, such description is generated automatically by default. Our application also allows to export all the generated descriptions and other metadata of the stored images. Such approach is especially beneficial for people working with large amount of photos, for example site owners while working with huge amount of images this service may help to save lot of time by automatically generating caption for every image.

In this application authenticated users can upload images, access uploaded images, and delete them. Each uploaded image is stored with a short description initially generated by a Neural Network model, however users can change this description at any moment. Additionally, application implements a text-based search among images based on their description.

The user can upload image in variety of formats: png, jpeg, jpg, webp, svg, or gif format. Users can upload images individually, in bulk (by selecting multiple images or folder on local computer), or by uploading a zip file containing images.

Users can also create albums (groups of images). Each image either do not belong to any album or belong to exactly one album. Images initially can be added to some album during upload, users can also request a Neural Nework model to suggest albums for uploaded images. Images can be moved between albums, and albums can be deleted or renamed.

Additionally it is possible to download all the metadata of all images either across whole account or from the selected folder.

2 Overview of similar solutions

We will overview 3 most popular web applications for photo storage: Google Photos, iCloud, and Flickr. During analysis they will be compared to the functionality of our application presented in Problem statement.

2.1 Google Photo

Google Photo is a leader in private photo storing and their organization.

The service automatically analyzes photos, identifying various visual features and subjects. Users can search for anything in photos, with the service returning results from three major categories: People, Places, and Things. The computer vision of Google Photos recognizes faces (not only those of humans, but pets as well), grouping similar ones together (this feature is only available in certain countries due to privacy laws); geographic landmarks (such as the Eiffel Tower); and subject matter, including birthdays, buildings, animals, food, and more. [1]

Advantages:

- Advanced search system

  As was stated before, one of the biggest advantage of Google Photos comparing to other competitors is its ability to analyze images using machine learning. It extracts the information about three major categories: People, Places, and Things. Such deep analysis allows users to efficiently find their photos using text-based search. Additionally, users can find photos with the same person on it or made at the same place.

  Our application also supports searching by text, however such search will apply only to image descriptions. It will not be possible to search by People, Places, or Things unless they was mentioned in such description.

- Ability to share photos with anyone else

  It is possible to give access to one or several photos or even whole album to another registered users. It is also possible to generate special link to the photo or some group of photos which will give access to them even to unregistered users.

  Our application will keep all photos completely private and will not support any sharing.

- Instruments for photo edit

  Google Photo have variety instruments for photo editing in their web interface. For example it is possible to crop the photo, change contrast, rotate, adjust brightness, etc. There are also another kind of instruments which allows to combine photos in animation, slide show video, or collage.

  In contrast to Google Photos, our application will not support any editing of the images.

Disadvanteges:

[1] https://en.wikipedia.org/wiki/Google_Photos

- Image captions are hidden and by default empty

  In order to see and interact with image caption, the user needs at selected image do additional step by clicking on "Information" icon. This approach can raise complications if the user wants to go through a bunch of photos seeing a description for each of them. In case user will want to add some description to the photo, the user needs to start from the blank field.

  Our application shows images together with their captions and each image initially saved with generated caption, which already makes work instead of user or at least gives a starting point for the user. Additionally, our application also allows user to export all the metadata including description of the selected images, which is impossible at Google Photo.

2.2 iCloud

iCloud is a cloud service from Apple for storing different files, however it is widely used for photo storage as well. The process of handling photos in iCloud is called "iCloud Photos". iCloud is especially popular among iPhone and Macintosh users.

Advantages:

- Photo sharing with registered users

  It is possible to give another user access to your image, however such user must also have account at apple.com, while Google Photo allows to share images also with unregistered users via link.

  As was stated before, our application keeps all photos completely private and do not support sharing in any way.

Disadvantages:

- Not focused on photos

  Since iCloud stores all kind of files its interface is completely inappropriate for interaction with images. It does not display images in Carousel mode, it is possible to either preview images as a small thumbnail before opening, or open image as a file in separate tab.

  While our application supports Carousel mode and has interface more oriented on images.

- Does not support some common formats

  The preview of some widely used image formats are not supported (for example webp and svg) and will be treated just as files of unknown type. It is not possible to preview them and the only way to see their content is to download them and open on local PC using suitable image viewer.

  While our application supports png, jpeg, jpg, webp, svg, and gif formats.

- No search

  There is no search in iCloud which is essential feature when working with huge amount of files.

- No caption for images

  The ability to add a description to any file is completely absent.

2.3 Flickr

Flickr is an image hosting and video hosting service, as well as an online community, founded in Canada and headquartered in the United States. The images a photographer uploads to Flickr go into their sequential "photostream", the basis of a Flickr account. All photostreams can be displayed as a justified view, a slide show, a "detail" view or a date stamped archive. Clicking on a photostream image opens it in the interactive "photopage" alongside data, comments and facilities for embedding images on external sites. Users may label their uploaded images with titles and descriptions, and images may be tagged, either by the uploader or by other users, if the uploader permits it. These text components enable computer searching of Flickr. [2]

Advantages:

- Access to a huge database of public photos from other users

  Flickr presents a social network for photo sharing. You can follow other people who share photos and share your own photos with others as well.

  While our application remain all user photos private.

- Supports different kinds of metadata

  Each image can have description, location, tags and some other metadata. In Flickr descriptions for images are used very often, this allows Flickr to have efficient search engine that finds all images that satisfy the search query based on their metadata.

  Our application supports only descriptions for the images and we use only this kind of metadata during search.

Disadvantages:

- Does not support some common formats

  Some widely used image formats are not supported in Flickr (for example webp and svg), it is not even possible to upload them to Flickr.

  Our application supports png, jpeg, jpg, webp, svg, and gif formats.

[2] https://en.wikipedia.org/wiki/Flickr

- There is no captioning recommendations

  During image uploading user needs to start from the empty field. While it would be more useful to generate image caption in advance, how our application does, it would make all work instead of user or at least will give user a starting point. Such feature is especially important when uploading a huge amount of images, because it is hard to add description to every of them and in case user will not want to add description, later it would be impossible to find them using text-based search.

Summary In contrast to other existing solutions, the proposed application allows user to work not only with images but also with their descriptions which are initially generated by the Neural Network, the user can also export all the metadata of the stored images. Unlike several other applications, our application will support svg and webp formats. From the other side, one of the main disadvantages of our application is lack of sharing functionality, however, it is not compulsory, especially considering that primarily goal of the application is private photo storage.

3 Program Structure

The program structure is divided into two main sections: backend and frontend. The backend section itself is further divided into "Machine learning algorithm", "Database structure", and "REST API structure".

The important configuration settings are saved in three configuration files in the root of the repository: "frontend_config.json", "backend_config.cfg", and Nginx configuration file "nginx.conf".

3.1 Backend

The backend part consists of the machine learning algorithm, database, and REST API. All code for the backend part is fully stored in the "backend" folder located at the root of the repository.

The uploaded images are stored in the folder "backend/images". The content of this directory should be accessible to the python REST API script.

3.1.1 Machine learning algorithm

The machine learning algorithm is responsible for generating a description for a given image and classifying it among user's albums. It will work on Python using the PyTorch framework.

The machine learning algorithm will consist from the following parts:

1. Image encoder – generates image embeddings for each image using the

CLIP model[3] supplied by the open_clip python library[4].

2. Several MLP layers are used to convert CLIP embeddings into GPT2 embeddings for the first 10 tokens passed to it.

3. Decoder to text – decodes embedding into the text, consisting from a couple of sentences, using GPT2[5] from the HuggingFace library[6].

4. Text encoder – encodes the album names into embeddings that are represented in the shared latent space with the Image encoder. Using cosine similarity, it compares album embeddings with image embeddings to determine the best album for the provided image. Text encoding is also performed using the CLIP model.

All code required for training MLP layers (second item) is stored in the "backend/train.py" file which, after training is done, stores the whole pretrained model in the "backend/model.pt" file. The code required for loading the pretrained model, generating a text description for the given image, and classifying it among albums is located in "backend/predict.py".

3.1.2 Database structure

The database is stored in the local file in "backend/database.db" using SQLite3 format. The database instance is called "photo_storage" and includes the following tables:

- users – keeps record of every registered user. It includes the following columns:
  - id - unique INT identificator of the user
  - username - TINYTEXT representing unique username chosen by the user during account registration
  - password - BINARY(16) md5 hash password from the user's account
- albums – keeps a record of every album for every user. It includes the following columns:
  - id - unique INT identificator of the album
  - user - INT identificator of the owner of the album, this is the same entry as id from the "users" table
  - name - TINYTEXT name of the album

[3]Mokady, Ron, Amir Hertz, and Amit H. Bermano. "Clipcap: Clip prefix for image captioning." arXiv preprint arXiv:2111.09734 (2021).

[4]https://github.com/mlfoundations/open_clip

[5]Radford, Alec, et al. "Language models are unsupervised multitask learners." OpenAI blog 1.8 (2019): 9.

[6]https://huggingface.co/docs/transformers/model_doc/gpt2

- photos – keeps a record of every photo. It includes the following columns:
  - id - unique INT identificator of the photo
  - album - INT identificator of the album to which photo belongs
  - image - TINYTEXT filename of the uploaded photo
  - caption - TEXT field for storing a description of the image
  - date - DATETIME when the image was uploaded

3.1.3 REST API structure

REST API is implemented on python using the Flask framework. The api accepts requests and returns responses only in the JSON format with the content type "application/json". There are 3 types of responses and their corresponding codes that can be returned by python script:

1. 400 Bad Request

   It is returned if either request content wasn't successfully parsed by the server, some required JSON entries are missing, or some JSON entry is in the wrong format. The response with code 400 is returned with the following content, where <ERROR> is replaced by the description of the error cause:

   ```
   {
       "error": "<ERROR>"
   }
   ```

2. 401 Unauthorized

   The request to every api endpoint, except "/user/login" and "/user/signup", must be submitted with the "Authorization" header containing a Bearer token, which informs the server whether the user is authorized to make the given request. In case this token was not provided, expired, or does not have enough permissions to perform the request, then the response with code 401 is returned with the following content where <AUTHENTICATED> is replaced by true if the provided authorization header is valid but does not have enough permissions to perform given request and false otherwise:

   ```
   {
       "authenticated": <AUTHENTICATED>
   }
   ```

3. 200 OK

   Otherwise, if the request was processed successfully, the response with code 200 is returned. The content of the response is in JSON format and includes entries depending on the endpoint. If the endpoint has nothing to return, then an empty JSON body is returned:

{}

Endpoints Api supports the following endpoints and request methods:

POST /user/signup Registers a new user.

Accepts only requests in the following form where $<USERNAME>$ is replaced by the user's username and $<PASSWORD>$ is replaced by the user's password:

```
{
    username: "<USERNAME>",
    password: "<PASSWORD>"
}
```

In case username is specified in the invalid format, already taken by another user, or password is specified in the wrong format, the response 400 Bad Request is returned with the corresponding error.

Otherwise, if the user is registered successfully, the Bearer token, which can be used in the "Authorization" header for further requests, is returned.

POST /user/login Logs in the existing user.

Accepts only requests in the following form where $<USERNAME>$ is replaced by the user's username and $<PASSWORD>$ is replaced by the user's password:

```
{
    username: "<USERNAME>",
    password: "<PASSWORD>"
}
```

In case there is no entry in the database with the specified username-password pair the response 401 Unauthorized response is returned.

Otherwise, if the user is authenticated successfully, the Bearer token, which can be used in the "Authorization" header for further requests, is returned.

UPDATE /user Updates user's information.

Accepts only requests in the following form where $<USERNAME>$ is replaced by new username of the user (use "" or old username directly to keep the old one) and $<PASSWORD>$ is replaced by new password of the user (empty if the old password must remain):

```
{
    username: "<USERNAME>",
    password: "<PASSWORD>"
}
```

In case username is specified in invalid format, already taken by another user, or password is specified in the wrong format, the response 400 Bad Request is returned with the corresponding error.

Otherwise, if the user updated its information successfully, the new Bearer token, which can be used in the "Authorization" header for further requests, is returned. The previous token is automatically expired.

DELETE /user Deletes the user's account.

If the operation was successful returns 200 OK with empty JSON. The Bearer token is automatically expired.

GET /albums Returns the list of all albums belonging to the user. The format of the response is the JSON list with each item having id of the album and album name property.

```
[
    {
        id: <ID of the first user's album>,
        name: <NAME of the first user's album>
    },
    ...
]
```

PUT /album Creates a new album.

Accepts only requests in the following form where $<NAME>$ is replaced by the name of the new album:

```
{
    name: "<NAME>"
}
```

In case name is specified in the invalid format or already exists, the response 400 Bad Request is returned with the corresponding error.

Otherwise, it returns id of the new album in the JSON format:

```
{
    id: <NEW ID>
}
```

UPDATE /album Updates album name.

Accepts only requests in the following form where $<NAME>$ is replaced by new name of the album and $<ID>$ is replaced by the id of such album:

```
{
    id: <ID>,
    name: "<NAME>"
}
```

In case name is specified in the invalid format, already exists, or album with the provided id does not exist, the response 400 Bad Request is returned with the corresponding error.

In case album exists, but does not belong to the user, still the response with 400 response code will be returned stating "Album not found.".

Otherwise, empty JSON is returned.

DELETE /album Deletes an album.

Accepts only requests in the following form where $<ID>$ is replaced by the id of the album:

```
{
    id: <ID>
}
```

If id does not exists or exists, but album does not belong to the user returns, then response with code 400 is returned, describing the errorin its body. Otherwise, response with code 200 is returned, with empty JSON in the body.

GET /photo?id=<ID>&offset=<OFFSET> Gets top 50 photos from the specified album with the given offset or top 50 photos from any album belonging to the user when $<ID>$ is not provided.

The $<ID>$ must be replaced with the valid album id. In case no album id is specified the endpoint will return all photos belonging to the user. The photos are sorted by date and only at most 50 photos can be retrieved by single request. If $<OFFSET>$ is not specified top 50 photos will be returned, otherwise endpoint returns top 50 photos with the given offset.

In case if any of two parameters have wrong format they will be ignored.

The response has a format of the JSON list where each element is represented by a JSON object with photo id, caption, album id where photo is located, and uploading date.

```
[
    {
        id: <PHOTO ID>,
        caption: <PHOTO CAPTION>,
        album: <ALBUM ID WHERE PHOTO IS LOCATED>,
        date: <UPLOADING DATE>
    },
    ...
]
```

POST /photo Returns metadata of the requested photos (ids, captions, uploading dates, album name).

Accepts only requests in the following form where $<IDs>$ is replaced by an array of integers representing identificators of the photos user wants to retrieve:

```
{
    ids: <IDs>
}
```

If $<IDs>$ is not an array of integers the 400 response is returned. In case of problems with individual photos they are ignored. After successful execution code 200 with the JSON array of photo metadata is returned.

```
[
    {
        id: <ID>,
        caption: <PHOTO CAPTION>,
        album: <ALBUM NAME>,
        date: <UPLOADING DATE>
    },
    ...
]
```

PUT /photo Uploads new photo.

Unlike other endpoints accepts requests in multipart/form-data encoding. The request must include image file that we want to upload in field "file". File must have jpeg, jpg, png, gif, or webp extensions and include the image with the corresponding format.

If uploading is successful returns 200 and JSON in the response body with id, caption, date, and album fields.

```
{
    id: <PHOTO ID>,
    caption: <PHOTO CAPTION>,
    album: <ALBUM ID>,
    date: <UPLOADING DATE>
}
```

UPDATE /photo Updates photo caption and album id for the specified photo id.

Accepts only requests in the following form where $<ID>$ is replaced by the photo id, $<CAPTION>$ is replaced by the new caption of the photo (old can be used), and $<ALBUM>$ is replaced by the new album id (old can be used):

```
{
    id: <PHOTO ID>,
    caption: "<CAPTION>",
    album: <ALBUM ID>
}
```

If photo id or album id does not exists or does not belong to the user, or caption has invalid format returns code 400, with the corresponding error. Otherwise 200 with empty JSON in the body.

DELETE /photo Deletes photo with the specified id.

Accepts only requests in the following form where $<ID>$ is replaced by the photo id to be deleted:

```
{
    id: <ID>
}
```

If photo id does not exists or does not belong to the user returns code 400, with the corresponding error. Otherwise, status 200 with empty JSON in the body.

GET /image/<ID> Returns image file with the specified image id.

$<ID>$ must be replaced with valid image id. If image is not found or does not belong to the user the response with status 400 and the corresponding error is returned.

Image file returned in a format specified in the $Content-Type$ header. One of four formats should be expected: JPEG, PNG, GIF, WEBP.

GET /search/<QUERY>?offset=<OFFSET> Gets top 50 photos satisfying the search query with the given offset.

The $<QUERY>$ must be replaced with the text search query. In case no query is specified, the endpoint will return 400 Bad Request response code with the corresponding error in the body. The photos are sorted by date and only at most 50 photos can be retrieved by a single request. If $<OFFSET>$ is not specified, top 50 photos will be returned, otherwise 50 photos starting from the given offset will be returned. If the offset is in the wrong format 400 Bad Request status will be returned with the corresponding error.

The endpoint will make search by finding the exact similarities between query and image captions in the database.

3.2 Frontend

HTML, CSS, and Javascript are used for the frontend part. Additionally, Bootstrap framework for CSS and Vue JS framework for Javascript are used.

Frontend consists of 4 web pages: Landing, Login, Sign up, Application.

The landing page is a simple page located at '/' that describes a service and suggests to either login or sign up.

The login and sign up pages, located at '/login' and '/signup' respectively, are very similar pages, that have the form where the user can input its username and password, and login or sign up. They also have links to each other, so that a user is able to navigate from login page to sign up page and vice-versa.

The application page handles the core functionality. It displays user's photos in gallery mode and also contains a side panel with an album list where users can select the album whose content they wish to view. When selected, the content of the gallery changes to the content of the selected album. There is

also a search bar at the top. Once the search is done, its results substitute the previously displayed content in the gallery. All images displayed in the gallery are sorted by the upload date, from newest to oldest. When user clicks on the image from the image gallery it is opened in full size and user can perform various operations on this image, such as deletion, caption modification, user can also download the image itself or its metadata (caption and album name stored in the JSON format). The application page can also display a small upload window where users can upload new images. New images are uploaded with pre-generated descriptions and chosen album, the user can immediately modify it in the uploading window, the uploaded images can be also removed in this window if they were uploaded by mistake. The application can also show settings window where user can update username and password, download metadata of every photo in the JSON format, or delete the account.